

NIFTY100 FINANCIAL INTELLIGENCE PLATFORM

Analyst Guide

Version 1.0

Prepared By

Anil Pachar

B.Tech Electrical Engineering

BlueStock Internship Project

2026

Technologies Used

- **Python**
 - **Pandas**
 - **NumPy**
 - **SQLite**
 - **Streamlit**
 - **Plotly**
 - **FastAPI**
 - **Pydantic**
 - **Pytest**
-

Project Type

Financial Data Analytics Platform

1. Project Overview

The Nifty100 Financial Intelligence Platform is a complete financial analytics application developed to analyze the financial performance of companies listed in the Nifty 100 index. The project combines data engineering, financial analytics, machine learning, dashboard visualization, and REST API development into a single integrated platform.

The application begins with an ETL pipeline that extracts financial information from multiple Excel datasets. The data is validated, cleaned, transformed, and stored inside a centralized SQLite database. Once the database is prepared, several analytics modules calculate financial ratios, growth metrics, valuation indicators, and capital allocation measures.

The processed information is presented through an interactive Streamlit dashboard where users can explore company performance, compare businesses, analyze sectors, and generate PDF reports. In addition, a FastAPI-based REST API exposes the processed financial data, allowing external applications to access company information programmatically.

The project follows a modular software architecture, making it easy to maintain, extend, and test. Automated unit tests, API tests, and data validation checks ensure reliability and accuracy throughout the application.

Major Objectives

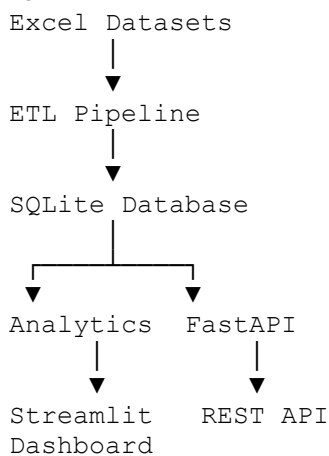
- Build a centralized financial database.**
- Automate the ETL process.**
- Generate financial ratios and valuation metrics.**
- Develop an interactive dashboard.**
- Implement company clustering using machine learning.**
- Expose financial data through REST APIs.**
- Ensure software quality using automated testing.**

2. System Architecture

The Nifty100 Financial Intelligence Platform is designed using a modular architecture where each component performs a specific task. This approach improves maintainability, scalability, and code reusability.

Financial data is collected from multiple Excel datasets and processed through an ETL pipeline. After validation and transformation, the cleaned data is stored in a centralized SQLite database. The analytics layer calculates financial ratios, valuation metrics, and clustering results. Finally, the processed data is presented through an interactive Streamlit dashboard and a FastAPI-based REST API.

System Architecture



Main Components

Data Layer: Stores financial data in a centralized SQLite database.

ETL Layer: Extracts, validates, transforms, and loads financial datasets.

Analytics Layer: Computes financial ratios, growth metrics, valuation measures, and clustering results.

Presentation Layer: Displays interactive dashboards using Streamlit.

API Layer: Provides REST endpoints for accessing financial data through FastAPI.

Benefits

- Modular design
- Easy maintenance
- Scalable architecture
- Faster data access
- Supports both Dashboard and REST AP

3. ETL Pipeline

The Nifty100 Financial Intelligence Platform follows an **Extract, Transform, and Load (ETL)** process to prepare financial data for analysis. This pipeline ensures that all datasets are cleaned, validated, and stored consistently before being used by the dashboard and REST API.

Extract

The extraction phase imports financial data from multiple Excel files. These datasets include company information, Profit & Loss statements, Balance Sheets, Cash Flow statements, financial ratios, market capitalization, sector details, and annual report links.

Transform

During the transformation stage, the raw data is processed to improve quality and consistency. Duplicate records are removed, missing values are handled, incorrect data types are corrected, and year formats are standardized. Financial metrics such as profitability ratios, valuation measures, and growth indicators are also calculated during this stage.

Load

After validation, the processed data is loaded into a centralized SQLite database. Each dataset is stored in its respective table, making data retrieval fast and organized.

Data Quality Checks

To maintain reliability, several validation checks are performed:

- Primary Key Validation
- Foreign Key Validation
- Duplicate Detection
- Missing Value Checks
- Data Type Validation
- Financial Consistency Checks

This structured ETL pipeline ensures that all analytical results displayed in the dashboard and API are accurate and consistent.

4. Database Design

The platform uses **SQLite** as its primary database because it is lightweight, reliable, and well suited for analytical applications. All processed financial data is stored in structured tables, allowing efficient querying and reporting.

The database contains information related to companies, financial statements, sector classification, market capitalization, financial ratios, and supporting documents.

Main Database Tables

- Companies
- Profit & Loss
- Balance Sheet
- Cash Flow
- Financial Ratios
- Market Capitalization
- Sector Information
- Analysis
- Documents
- Pros & Cons

Each table is connected through a common company identifier, ensuring consistency across the entire database.

Database Benefits

- Centralized storage of financial data
- Fast SQL-based queries
- Reduced data duplication
- Easy integration with Python and FastAPI
- Efficient support for dashboard visualizations

The database acts as the foundation of the platform, supplying validated information to the analytics engine, dashboard, and REST API.

5. Dashboard Guide

The Streamlit dashboard provides an interactive interface for exploring financial information of Nifty 100 companies. It is designed to help analysts quickly access important metrics through visualizations and filters.

Dashboard Modules

Home: Displays project overview and summary statistics.

Company Profile: Shows company information, financial ratios, valuation metrics, CAGR values, and business strengths and weaknesses.

Stock Screener: Filters companies based on parameters such as Return on Equity (ROE), Debt-to-Equity ratio, Revenue CAGR, Profit CAGR, Price-to-Earnings (P/E) ratio, and sector.

Peer Comparison: Compares multiple companies using interactive charts and financial metrics.

Trend Analysis: Visualizes historical trends for revenue, profit, earnings per share (EPS), operating margins, and other financial indicators.

Sector Analysis: Provides sector-wise performance comparison and key financial statistics.

Capital Allocation: Evaluates capital efficiency using metrics such as ROE, ROCE, Debt-to-Equity ratio, and Cash Flow.

Annual Reports: Allows users to browse and access annual report links for individual companies.

The dashboard is fully interactive and enables users to perform financial analysis efficiently through charts, tables, and filtering options.

6. Financial Analytics

The Financial Analytics module transforms raw financial statements into meaningful business insights. Using Python and Pandas, the platform calculates important financial indicators that help evaluate a company's profitability, growth, valuation, and financial stability.

Key Financial Metrics

Return on Equity (ROE)

Measures how efficiently a company generates profit using shareholders' equity.

Operating Profit Margin (OPM)

Shows the percentage of revenue remaining after operating expenses.

Net Profit Margin (NPM)

Represents the percentage of revenue converted into net profit.

Debt-to-Equity Ratio

Evaluates the company's financial leverage and long-term solvency.

Interest Coverage Ratio (ICR)

Measures the company's ability to pay interest expenses from operating earnings.

Revenue CAGR

Shows the annual growth rate of revenue over a five-year period.

Profit CAGR

Measures long-term profit growth.

These metrics allow investors and analysts to compare companies objectively and identify financially strong businesses.

7. Machine Learning

The project includes a Machine Learning module that groups companies based on their financial characteristics. K-Means Clustering is used to identify companies with similar financial behaviour.

Five important financial features are selected before clustering:

- Return on Equity
- Debt-to-Equity Ratio
- Revenue CAGR
- Free Cash Flow CAGR
- Operating Profit Margin

Before training the model, missing values are handled and all features are standardized using StandardScaler. The elbow method is applied to determine the optimal number of clusters.

Each company is assigned to one of five financial groups. Cluster profiling is then performed to understand the characteristics of every cluster.

Benefits of Clustering

- Identifies similar companies.
- Supports investment research.
- Helps compare companies beyond sectors.
- Simplifies portfolio analysis.
- Provides deeper financial insights.

The clustering results are exported as CSV files and can also be accessed through the REST API.

8. REST API Guide

The platform provides a REST API developed using FastAPI. The API allows external applications to access financial information stored in the SQLite database.

Swagger UI automatically generates interactive documentation, while ReDoc provides detailed API documentation for developers.

Main Endpoints

Endpoint	Purpose
GET /	Home
GET /health	Health Check
GET /companies	List all companies
GET /companies/{company_id}	Company details
GET /financials/{company_id}	Financial ratios
GET /clusters	Company clusters
GET /clusters/summary	Cluster summary
GET /clusters/sectors	Sector distribution
GET /clusters/top-companies	Top companies
GET /validate	Validate resources
GET /info	Project information

Example API Calls

```
curl http://127.0.0.1:8000/companies
```

```
curl http://127.0.0.1:8000/companies/TCS
```

```
curl http://127.0.0.1:8000/financials/TCS
```

The API is fully documented and can be tested directly using Swagger.

9. Testing and Troubleshooting

The project was tested at different stages to ensure correctness, reliability, and stability.

Testing Performed

- ETL Validation
- Data Quality Checks
- Financial Ratio Validation
- API Testing
- Dashboard Testing
- Integration Testing
- Unit Testing using Pytest

The API endpoints were verified to ensure correct responses and proper error handling. Automated tests were executed after every major implementation phase to maintain software quality.

Troubleshooting

Issue	Solution
Database not found	Verify db/nifty100.db exists
Dashboard not loading	Install dependencies using requirements.txt
API not starting	Start the server using Uvicorn
Swagger not opening	Ensure port 8000 is available
Missing output files	Run the ETL and analytics modules again

Following these practices helps maintain smooth project execution and reduces runtime errors.

10. Conclusion

The Nifty100 Financial Intelligence Platform demonstrates a complete financial analytics workflow, beginning with data extraction and ending with interactive dashboards, machine learning insights, and REST API services.

Throughout the project, multiple technologies were integrated to build a scalable and maintainable application. SQLite provides centralized data storage, Pandas performs data processing, Streamlit delivers interactive dashboards, FastAPI exposes REST endpoints, and Pytest ensures software quality through automated testing.

The modular design allows new datasets, analytical models, and dashboard features to be added with minimal changes to the existing codebase.

Project Achievements

- Developed a complete ETL pipeline.
- Created a centralized SQLite database.
- Built an interactive Streamlit dashboard.
- Implemented financial analytics and valuation metrics.
- Generated automated PDF reports.
- Applied K-Means clustering for company analysis.
- Developed REST APIs using FastAPI.
- Performed automated testing for project validation.

This project serves as a practical demonstration of financial data engineering, analytics, visualization, API development, and software testing. It provides a strong foundation for future enhancements such as cloud deployment, authentication, real-time market data integration, and advanced machine learning models.